BookVerseDB.Authors.json


```
[{
  "_id": {
    "$oid": "69079b20dda5fb3b80596092"
  },
  "name": "shruti",
  "nationality": "indian",
  "birthYear": 2002
},
{
  "_id": {
    "$oid": "69079b20dda5fb3b80596093"
  },
  "name": "jony",
  "nationality": "america",
  "birthYear": 2000
},
{
  "_id": {
    "$oid": "69079b20dda5fb3b80596094"
  },
  "name": "don",
  "nationality": "russia",
  "birthYear": 1995
}]
```

BookVerseDB.Books.json


```
[{
  "_id": {
    "$oid": "6907a60a1ee35d232989d5ca"
  },
  "title": "1984",
  "genre": "Dystopian",
  "publicationYear": 1949,
  "authorId": {
    "$oid": "69079b20dda5fb3b80596092"
  },
  "ratings": [
    {
      "user": "vishu",
      "score": 5,
      "comment": "A chilling masterpiece."
    }
  ]
},
{
  "_id": {
    "$oid": "6907a60a1ee35d232989d5cb"
  },
  "title": "Animal Farm",
  "genre": "Political Satire",
  "publicationYear": 1945,
```

```
  "authorId": {
    "$oid": "69079b20dda5fb3b80596093"
  },
  "ratings": [
    {
      "user": "yash",
      "score": 5,
      "comment": "Brilliant allegory."
    }
  ]
},
{
  "_id": {
    "$oid": "6907a60a1ee35d232989d5cc"
  },
  "title": "Harry Potter and the Sorcerer’s Stone",
  "genre": "Fantasy",
  "publicationYear": 1997,
  "authorId": {
    "$oid": "69079b20dda5fb3b80596092"
  },
  "ratings": [
    {
      "user": "shruti",
      "score": 5,
      "comment": "Magical and unforgettable."
```

```
    }
  ]
}]
```


BookVerseDB.Users.json


```
[{
  "_id": {
    "$oid": "6907a59e5570d2c26fdde9dc"
  },
  "name": "shruti",
  "email": "shruti@example.com",
  "joinDate": {
    "$date": "2023-06-15T00:00:00.000Z"
  }
},
{
  "_id": {
    "$oid": "6907a59e5570d2c26fdde9dd"
  },
  "name": "yash",
  "email": "yash@example.com",
  "joinDate": {
    "$date": "2024-01-10T00:00:00.000Z"
  }
},
```

```
{
  "_id": {
    "$oid": "6907a59e5570d2c26fdde9de"
  },
  "name": "harsh",
  "email": "harsh@example.com",
  "joinDate": {
    "$date": "2024-09-05T00:00:00.000Z"
  }
}]
```


authors.js

```
db.Authors.insertMany([
  { name: "shruti", nationality: "indian", birthYear: 2002 },
  { name: "jony", nationality: "america", birthYear: 2000 },
  { name: "don", nationality: "russia", birthYear: 1995 }
]);
```

Book.js

```
db.Books.insertMany([
  {

    "title": "1984",
    "genre": "Dystopian",
    "publicationYear": 1949,
    "authorId": ObjectId("69079b20dda5fb3b80596092"),
    "ratings": [
```

```
      { "user": "Alice Johnson", "score": 5, "comment": "A chilling masterpiece." }
    ]
  },
  {

    "title": "Animal Farm",
    "genre": "Political Satire",
    "publicationYear": 1945,
    "authorId": ObjectId("69079b20dda5fb3b80596093"),
    "ratings": [
      { "user": "Clara Lee", "score": 5, "comment": "Brilliant allegory." }
    ]
  },
  {

    "title": "Harry Potter and the Sorcerer’s Stone",
    "genre": "Fantasy",
    "publicationYear": 1997,
    "authorId": ObjectId("69079b20dda5fb3b80596092"),
    "ratings": [
      { "user": "Alice Johnson", "score": 5, "comment": "Magical and unforgettable." }
    ]
  }
]
);
```

```
// create index
db.Books.createIndex({ genre: 1 });
db.Books.createIndex({ authorId: 1 });
db.Books.createIndex({ "ratings.score": 1 });

// getindex
db.Books.getIndexes();

// after creating index
db.Books.find({ genre: "Fantasy" }).explain("executionStats");

//agreagte
db.Books.aggregate([
  { $unwind: "$ratings" },
  {
    $group: {
      _id: "$_id",
      title: { $first: "$title" },
      avgRating: { $avg: "$ratings.score" }
    }
  }
]);

// Retrieve the top 3 highest-rated books
```

```
db.Books.aggregate([
  {
    $group: {
      _id: "$_id",
      title: { $first: "$title" },
      avgRating: { $avg: "$ratings.score" }
    }
  },
  { $sort: { avgRating: -1 } },
  { $limit: 3 }
]);

// Count the number of books published per genre
db.Books.aggregate([
  {
    $group: {
      _id: "$genre",
      totalBooks: { $sum: 1 }
    }
  },
  { $sort: { totalBooks: -1 } }
]);
// Display the total reward points (sum of all ratings) received by each author

db.Books.aggregate([
```

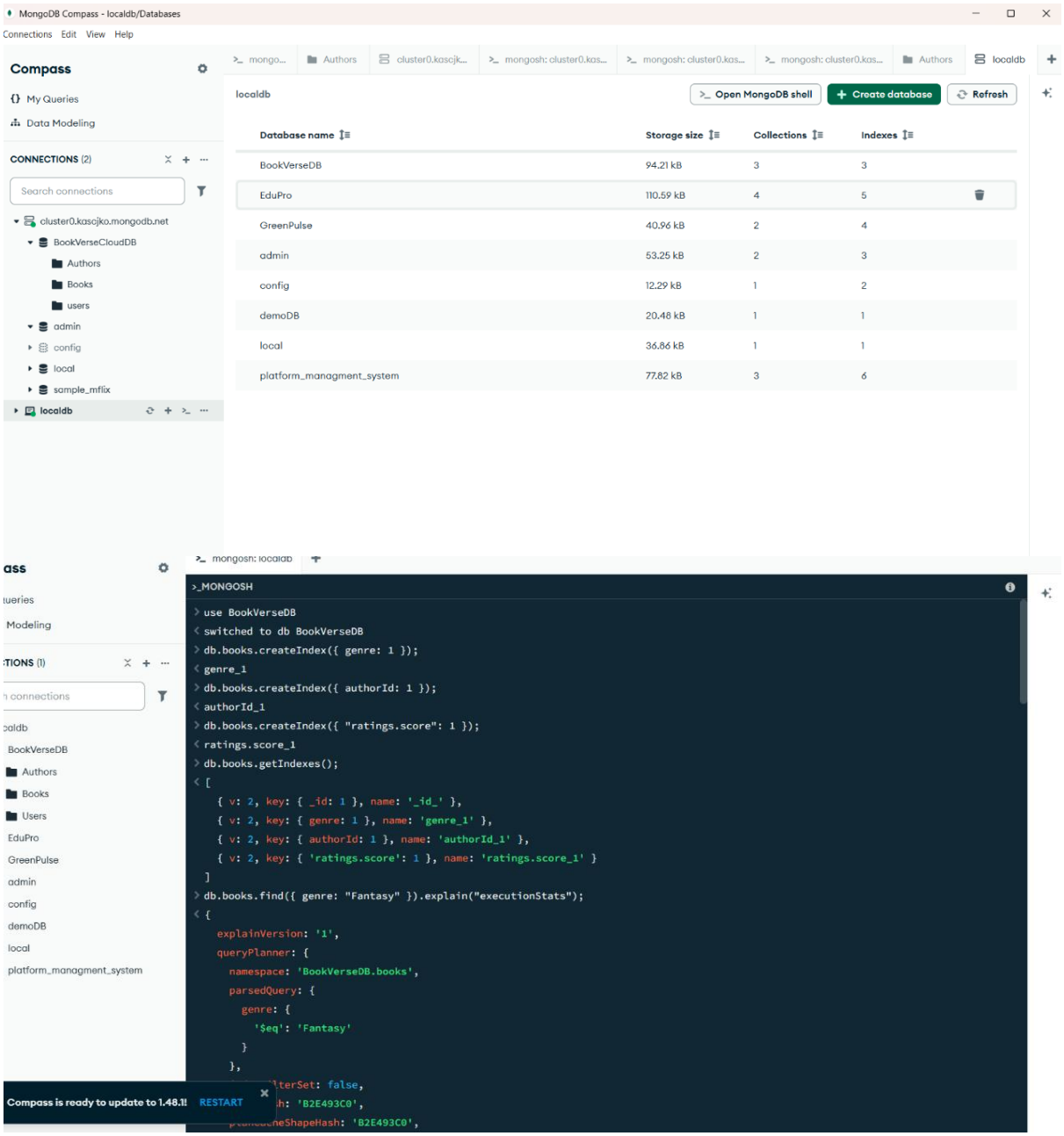
```
  {
    $group: {
      _id: "$authorId",
      totalPoints: { $sum: "$ratings.score" }
    }
  }
]);
```

User.js

```
db.Users.insertMany([
  {

    "name": "shruti",
    "email": "shruti@example.com",
    "joinDate": ISODate("2023-06-15T00:00:00Z")
  },
  {

    "name": "yash",
    "email": "yash@example.com",
    "joinDate": ISODate("2024-01-10T00:00:00Z")
  },
  {

    "name": "harsh",
    "email": "harsh@example.com",
```

```
    "joinDate": ISODate("2024-09-05T00:00:00Z")
  }
]
);
```

Screenshot


MongoDB Compass - localdb/Databases
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (2)
Search connections
cluster0.kascjko.mongodb.net
BookVerseCloudDB
Authors
Books
users
admin
config
local
sample_mflix
localdb
mongo...
Authors
cluster0.kascjk...
mongosh: cluster0.kas...
mongosh: cluster0.kas...
mongosh: cluster0.kas...
Authors
localdb
localdb
Open MongoDB shell
Create database
Refresh
Database name
Storage size
Collections
Indexes
BookVerseDB 94.21 kB 3 3
EduPro 110.59 kB 4 5
GreenPulse 40.96 kB 2 4
admin 53.25 kB 2 3
config 12.29 kB 1 2
demoDB 20.48 kB 1 1
local 36.86 kB 1 1
platform_managment_system 77.82 kB 3 6
mongosh: localdb
>_MONGOSH
use BookVerseDB
switched to db BookVerseDB
db.books.createIndex({ genre: 1 });
genre_1
db.books.createIndex({ authorId: 1 });
authorId_1
db.books.createIndex({ "ratings.score": 1 });
ratings.score_1
db.books.getIndexes();
[
{ v: 2, key: { _id: 1 }, name: '_id_' },
{ v: 2, key: { genre: 1 }, name: 'genre_1' },
{ v: 2, key: { authorId: 1 }, name: 'authorId_1' },
{ v: 2, key: { 'ratings.score': 1 }, name: 'ratings.score_1' }
]
db.books.find({ genre: "Fantasy" }).explain("executionStats");
{
explainVersion: '1',
queryPlanner: {
namespace: 'BookVerseDB.books',
parsedQuery: {
genre: {
'$eq': 'Fantasy'
}
},
Compass is ready to update to 1.48.1
RESTART